Intro to Machine Learning

EECS 492

Introduction to Artificial Intelligence

Reminders

- HW 1 to be released today
  - Start early!
  - Topics: Agents and ML
  - Coding resources available in Canvas
    - Files > Coding Resources
    - Includes video guide on virtual environments
- HW1 due date: Feb 4

Today's Class

- Unsupervised learning
- Supervised learning

Learning

- How can we tell if an agent is learning?
  - Improves performance on future tasks after making observations about the world

- Why is learning desirable?
  - Can designers anticipate all possible situations?
  - Can designers anticipate change?
  - What if we, as people, have no idea how to structure the task?

Types of Feedback

- Reinforcement learning
  - Agent learns from series of reinforcements (rewards/punishments)
  - Builds off of the MDP/POMDP formalism
- Unsupervised learning
  - No explicit feedback is supplied
  - Most common method: clustering
- Supervised learning
  - Agent observes example input-output pairs
  - Learns functions mapping input to output

Also very popular:
Self-supervised learning!

Unsupervised Learning

Unsupervised Learning

- Unsupervised learning – agent learns patterns in the input with no explicit feedback
- Most common task is clustering – detecting potentially useful clusters of input examples
- Example: given millions of images from the Internet, a system can identify large clusters of similar images
  - E.g., Maybe it will find a large cluster of images that look like cats

k-means Clustering

- Let's say we have **n** observations (data points) that we want to group into **k** clusters
  - Each cluster has a centroid – the mean point of the cluster
  - We want each data point to belong to the cluster with the nearest centroid

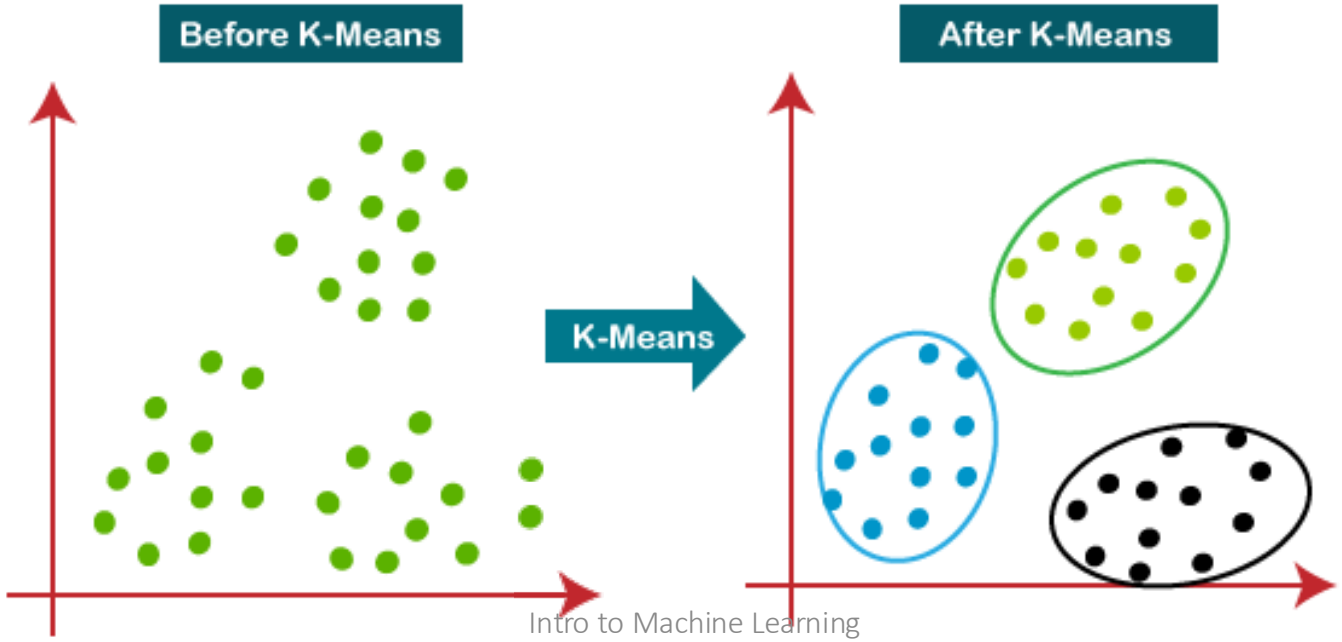


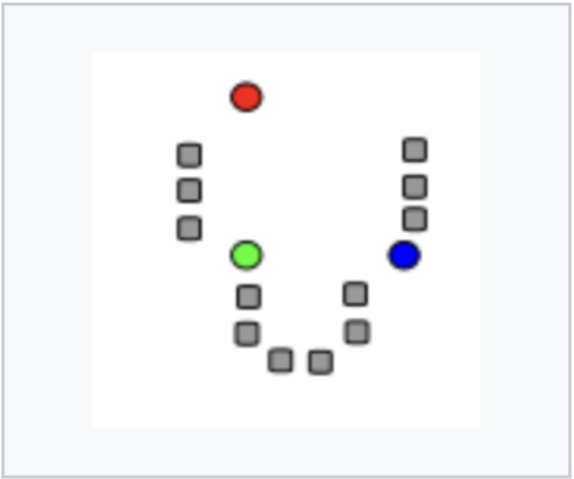
Image from https://www.javatpoint.com/k-means-clustering-algorithm-in-machine-learning

k-means Clustering Algorithm

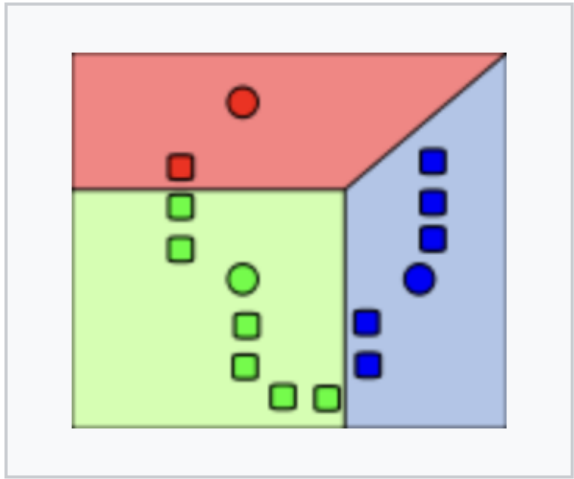
- Lloyd's algorithm
- Uses iterative refinement

- Start by randomly choosing k points to be centroids
- Then iterate:
  - Assign each data point to the cluster with the nearest centroid
  - Now, recalculate each centroid (take the average of all the points in the cluster)
- Stop iterating when the cluster assignments no longer change

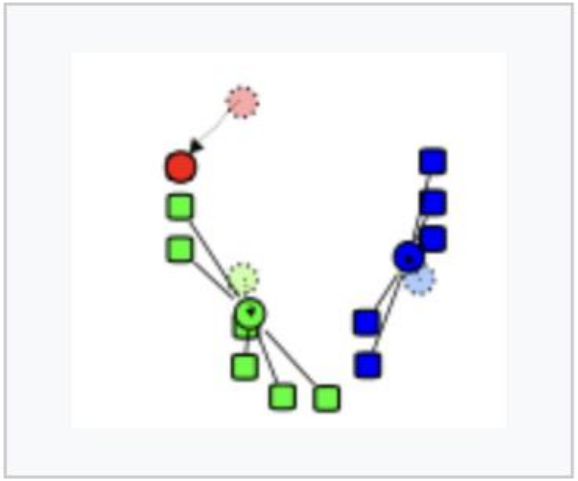
k-means Clustering Algorithm



Randomly choose 3 points to be centroids



Assign each data point to the cluster with the closest centroid



Update the centroids

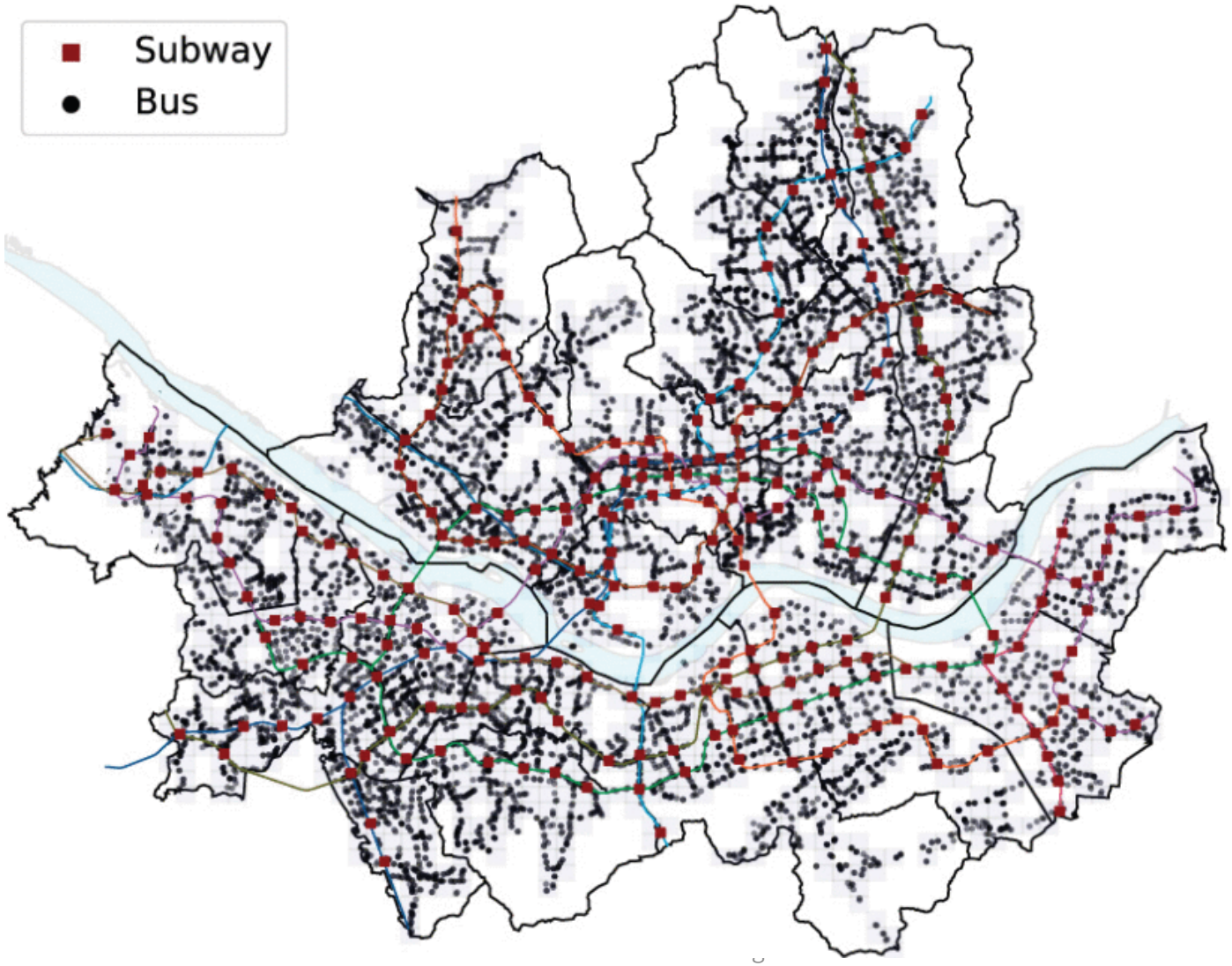
Repeat until convergence

Image from Wikipedia

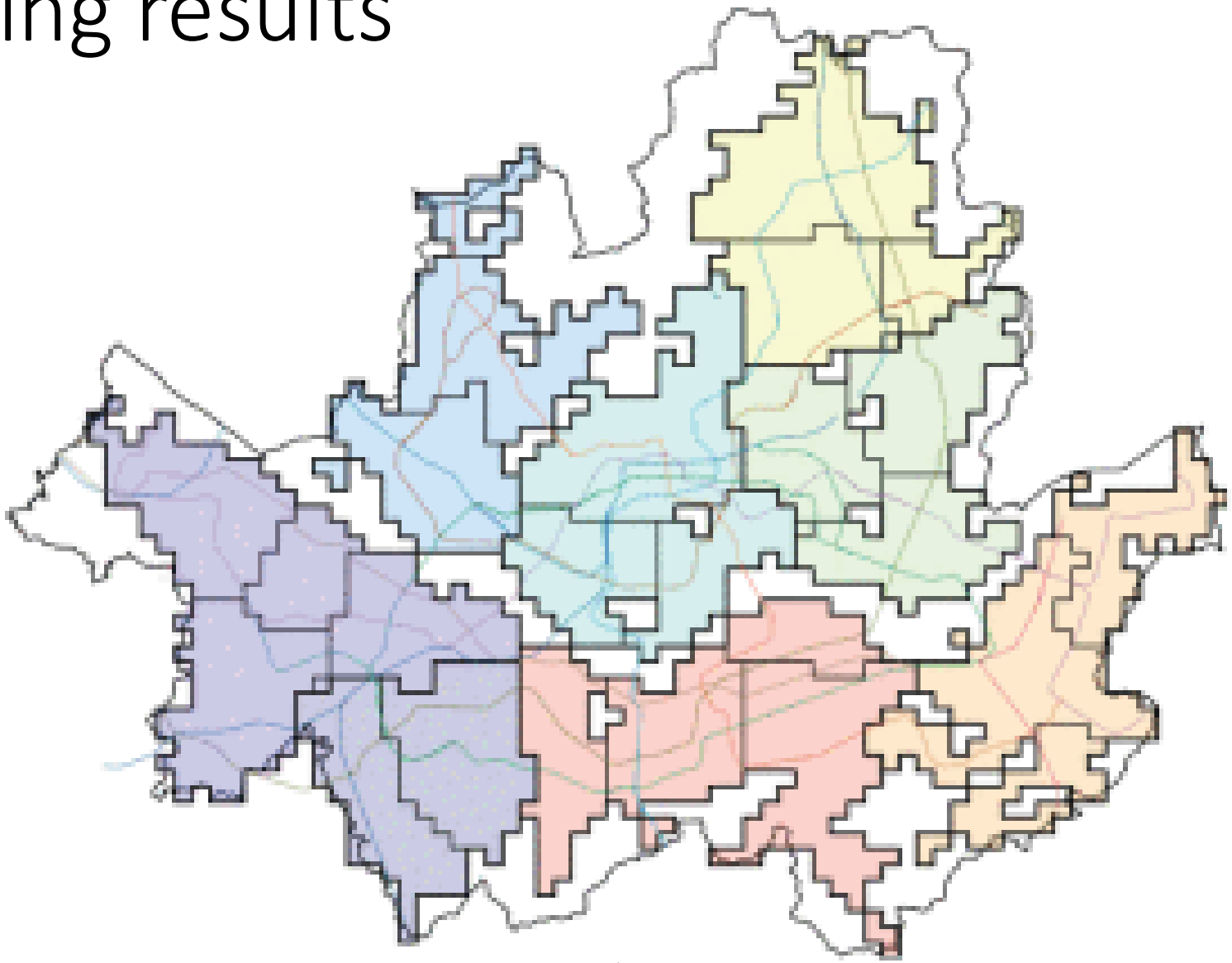
Real-world Clustering Example

- Paper: “Identifying the Structure of Cities by Clustering Using a New Similarity Measure Based on Smart Card Data” (on Canvas)
- “In metropolises, essential resources, infrastructures, and facilities are allocated throughout cities on the basis of urban planning. Different regions may play various roles or functions in cities. For example, certain areas are primarily responsible for residents, whereas others are specialized for commercial activities. Such urban forms are closely related to intra-city travel patterns; therefore, the interdependence between urban forms and travel behaviors has been examined for a long time in urban studies…”

Subway map of Seoul, South Korea



Clustering results



Supervised Learning

Supervised Learning

- Problem set-up:
  - Given: training set of N example input-output pairs

$$(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)$$

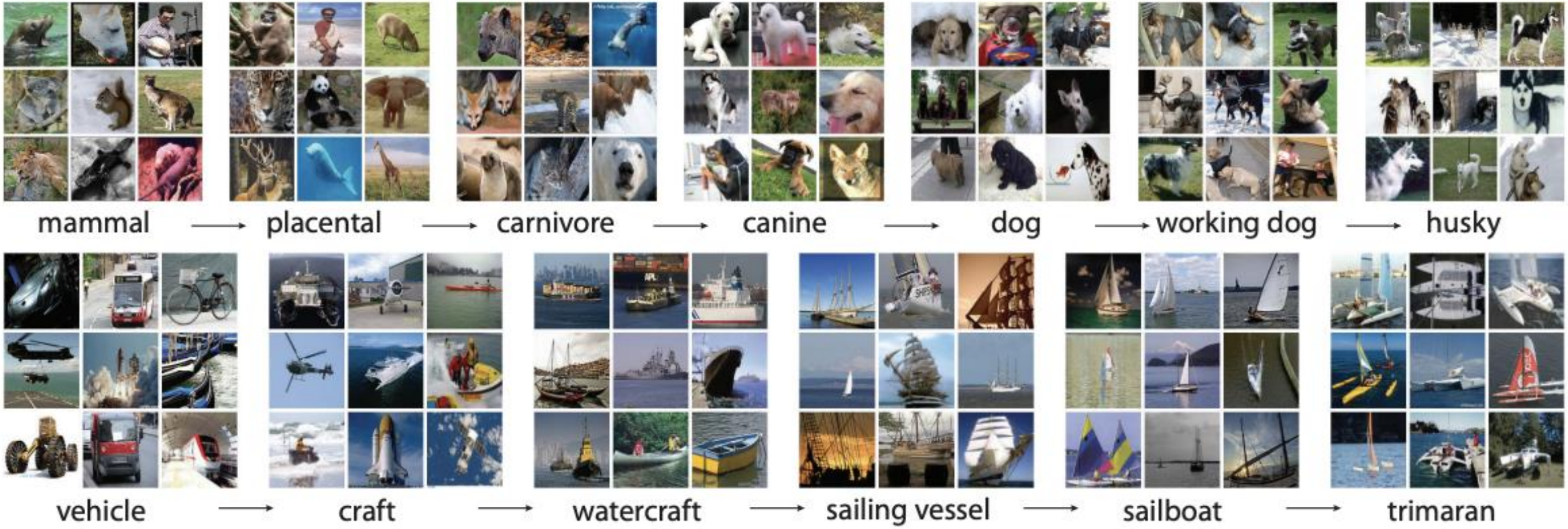
Each y_j was generated by an unknown function $y = f(x)$

  - Goal: discover function, *h*, approximating true function, *f*

- Learning is search through the set of possible hypotheses (*h*)

Example: ImageNet

- ImageNet is a dataset of > 1.2 million images with labels



Supervised Learning

ImageNet: training set consists of (image, label) pairs

- Problem set-up:
  - Given: training set of N example input-output pairs

$$(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)$$

We want to approximate some unknown function *f* that maps from images to labels

Each y_j was generated by an unknown function $y = f(x)$

  - Goal: discover function, *h*, approximating true function, *f*

- Learning is search through the set of possible hypotheses (*h*)

Once we have approximated *f*, we can give our system a new image, and it can label an image it's never seen before!

Building a ML Model

- **Training Time**
  - Use labeled input/output examples to learn a machine learning model
    - E.g., learn parameters of model

- **Deployment Time**
  - Run trained model on examples where we don’t know the output

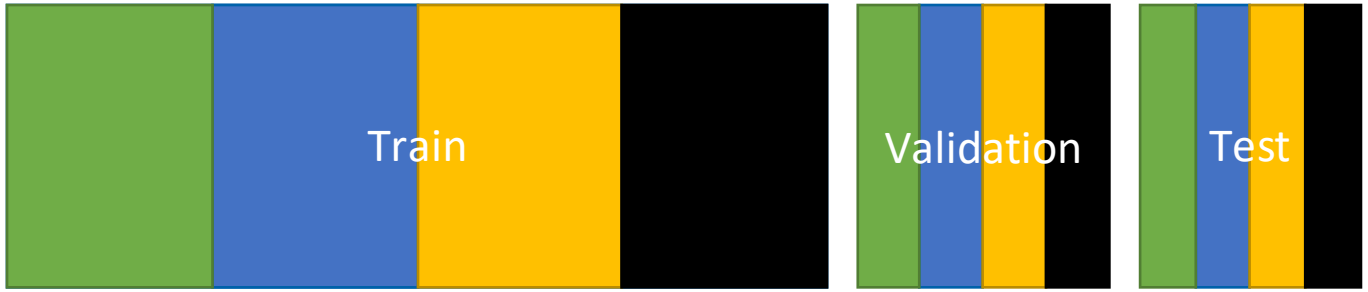
Validating Models

- **Data separation**
  - Training set – used to learn the models
  - Validation set – used to adjust parameters of the models
  - Testing set – used to validate the models
  - Method performs well if models generalize to testing set

- Data modeling
  - Classification – assign a discrete label to a data point (e.g., predict if the weather will be sunny)
  - Regression – estimate a numeric value associated with that data point (e.g., predict energy efficiency of a car)
  - Prediction vs. data fitting

Train/validation/test

Train/validation/test



Key:

Train/validation/test

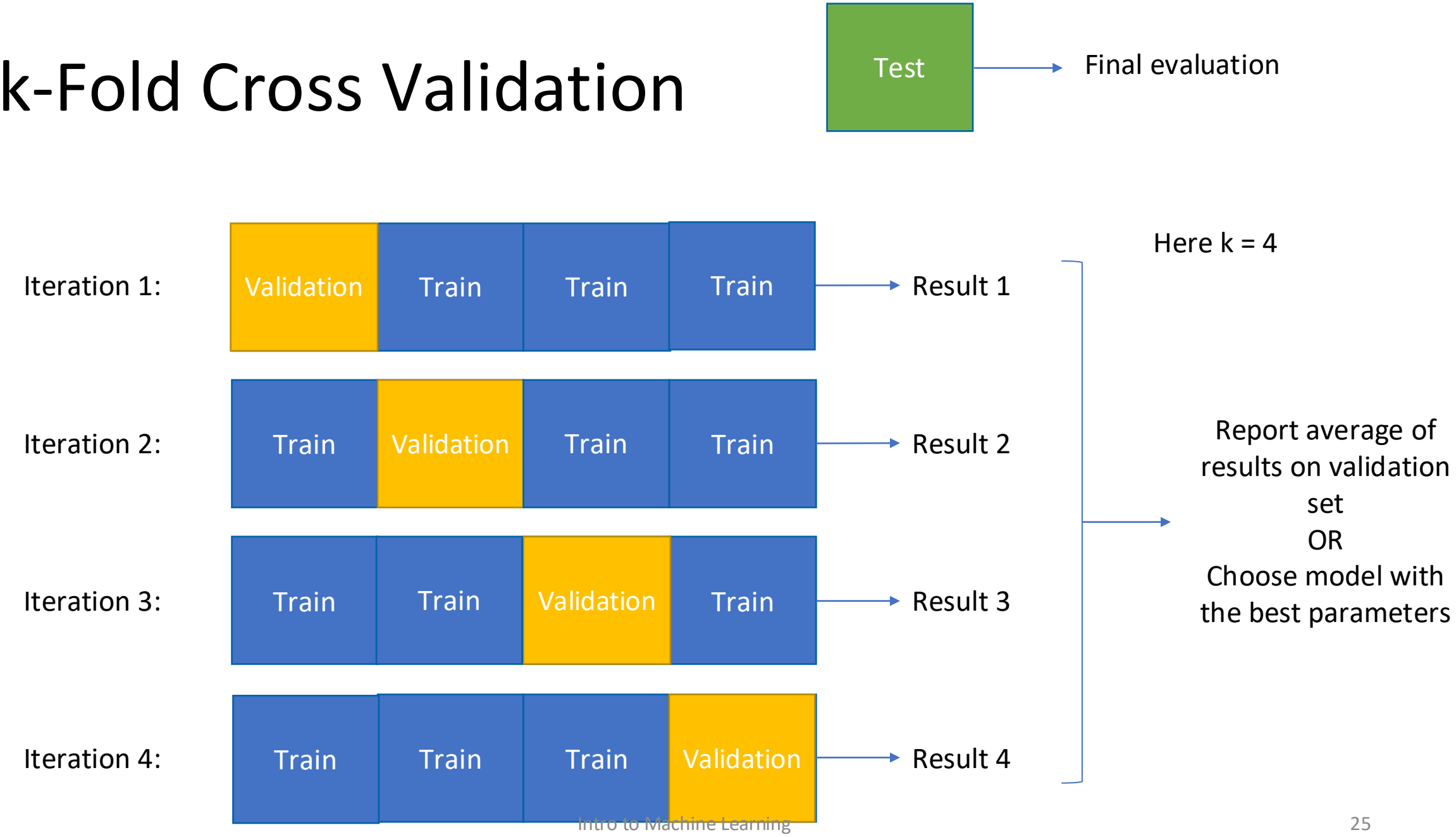
Key:

k-Fold Cross Validation

k-Fold Cross Validation

Here k = 4

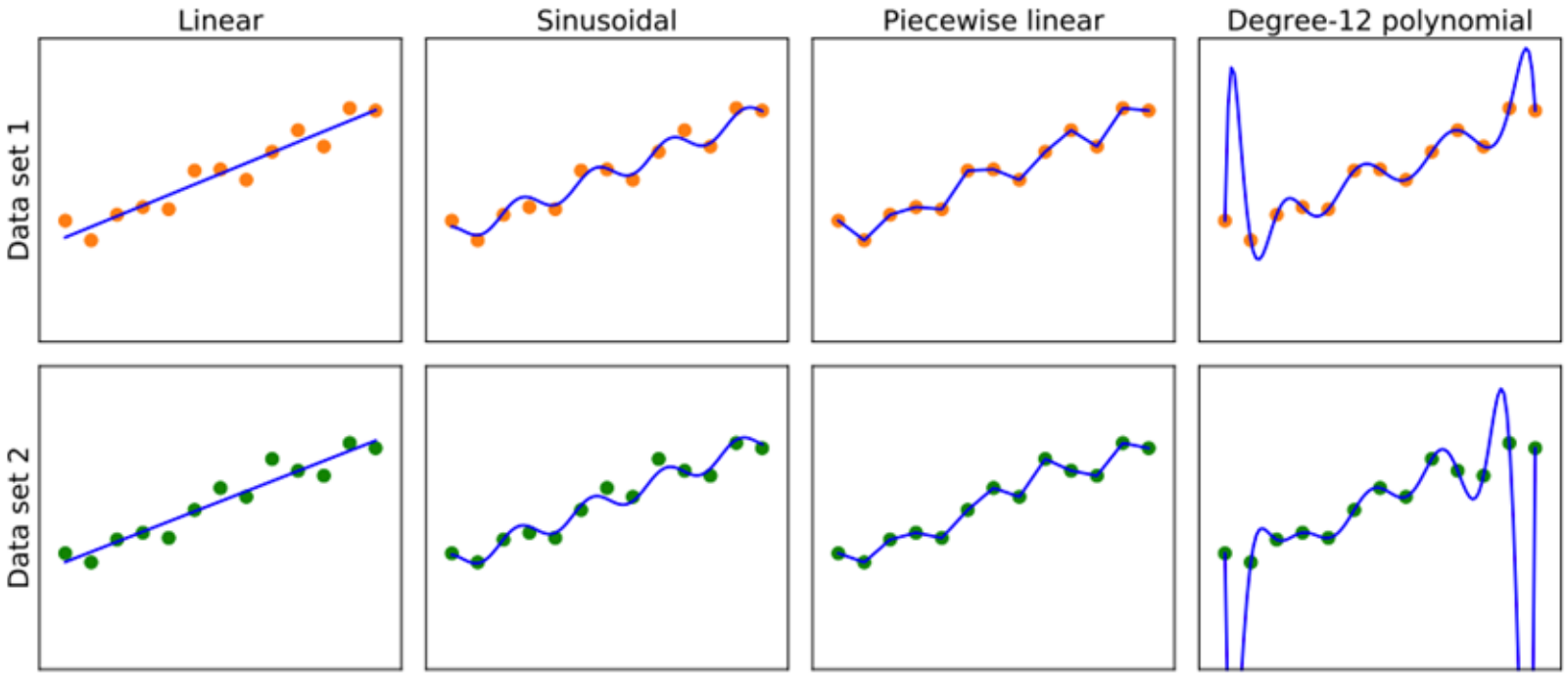
k-Fold Cross Validation



Bias-Variance Trade-off

- Bias – (loosely) how much error the model has on the training data
- Variance – how much the model changes due to fluctuations in the training data

Bias-Variance Trade-off



High bias (doesn't fit the data as well)

Low variance (model is pretty much the same on two similar training data sets)

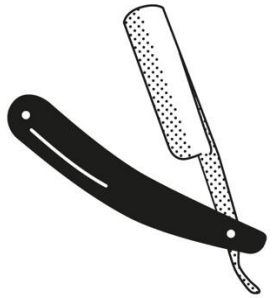
Low bias (fits the data really well)

High variance (model changes dramatically on two similar training data sets!)

Bias-Variance Trade-off

- Trade-off:
  - More complex (high variance) models that fit the training data well (low bias)
  - Simpler (low variance) models that don't fit the training data as well (high bias)
  - Usually can't have both low bias and low variance!

- Ockham's razor: choose the simplest model that matches the data
  - Simplicity alone doesn't guarantee correctness, but it often leads to more useful and understandable solutions
  - Ockham's razor promotes simplicity as a heuristic guide for model selection



- But watch out for underfitting and overfitting
  - Underfitting – model fails to find a pattern in the data
  - Overfitting – model pays too much attention to a particular dataset and can't generalize to unseen data